CC JS : Session 1 - 1h30 - Sur machine

Téléchargez l'archive **cc-20-js-sujet.tgz** de l'espace Moodle. Après extraction, renommez le répertoire obtenu **nom-prenom** où **nom** et **prenom** sont vos nom et prénom écrits en minuscules et sans espace (par ex. **delafontaine-jean**). A l'issue de l'examen, archivez ce répertoire (`tar czf nom-prenom.tgz nom-prenom`) et déposez l'archive sur Moodle.

Les exercices sont indépendants et peuvent être traités dans l'ordre de votre choix. Pour visualiser ce qui est attendu, consultez ce démonstrateur.

Exercice 1. (**Tableaux et objets**)
Le dossier **exercice-1** contient le fichier **exercice1.html** qui importe le fichier **exercice1.js**. Ce dernier déclare une variable `villes49` qui est un tableau d'objets dotés des mêmes noms de propriétés et représentant chacun une des communes du département. Vous trouverez en fin du fichier Javascript les parties à compléter pour chaque question et le résultat attendu dans la console du navigateur. Vous testerez votre code en chargeant le fichier HTML et en visualisant les résultats affichés dans la console.
1. A partir de `villes49`, calculez le nombre de communes dont la population en 2012 était supérieure ou égale à 1000 habitants. Stockez le résultat dans la variable `nr_villes_1K` qui sera affichée à la console.
2. A partir de `villes49`, construisez le tableau des cantons des communes, sans doublons (un canton n'apparaît qu'une fois). Stockez le résultat dans la variable `cantons` dont la taille sera affichée à la console.
3. A partir de `villes49`, construisez le tableau `villes_denses` des 4 communes de plus forte densité de population. Chaque élément de `villes_denses` sera un tableau à 2 éléments contenant le nom de la commune et sa densité. Les 4 communes apparaîtront par ordre de densité décroissante. `villes_denses` sera affiché à la console.
4. A partir de `villes49`, calculez l'écart maximum entre l'altitude minimum et l'altitude maximum d'une commune. Stockez le résultat dans la variable `dénivellation` qui sera affichée à la console.
5. Le constructeur `CodePostal` vous est fourni. Il permet de créer des objets stockant un code postal (entier) passé en argument du constructeur. A partir de `villes49`, construisez le tableau `villes` des objets construits avec `CodePostal` et contenant chacun le code postal d'une commune. Ajoutez ensuite à `CodePostal`, sans le modifier, une méthode `departement` qui, lorsqu'elle est invoquée sur un objet, renvoie l'entier correspondant aux 2 premiers chiffres du `code` de l'objet, autrement dit, le numéro de département associé au code postal. La variable booléenne `maineetloire`, qui est affichée à la console, permet alors de tester si toutes les communes répertoriées dans `villes49` sont bien du département 49.

Exercice 2. (**Construction d'un DOM**)
Le dossier **exercice-2** contient les fichiers **exercice2.html** et **exercice2.js**. Vous devrez compléter le fichier **exercice2.js** avec le code pour les fonctions genererTableau() et supprimerOutput(), sans modifier le fichier html.

1. La fonction `genererTableau()` génère le code HTML nécessaire pour :
   (a) Ajouter un texte en gras avec la phrase *"Tableau généré le <date_actuelle>"*. Pour obtenir la date/heure actuelle, vous pouvez utiliser le code ci-dessous :
   ```
   var d = new Date();
   d.toString();
   ```
   (b) Ajouter un tableau de taille égal aux nombres de lignes et colonnes saisis à partir des éléments <input> (voir fichier html). Chaque cellule du tableau contiendra un nombre entier aléatoire, à générer avec les fonctions `Math.round(Math.random() * 100)`). Les nombres aléatoires peuvent être répétés.
   (c) Ajouter deux sauts de ligne entre le texte et le tableau.
   (d) Si le nombre de lignes ou colonnes est inférieure à 1, la fonction affichera un paragraphe (en couleur rouge) avec la phrase : *"Erreur ! Vérifiez que le nombre de lignes ou colonnes soit supérieur à zero."*
2. La fonction `supprimerOutput()` doit supprimer tous les éléments créés par la fonction `genererTableau()`.

Exercice 3. (**Modification statique d'un document HTML**)
Dans cet exercice vous modifierez **uniquement tux.js** dans le but de modifier automatiquement l'affichage du document HTML **tux.html**. La Figure 1 montre le résultat à obtenir.

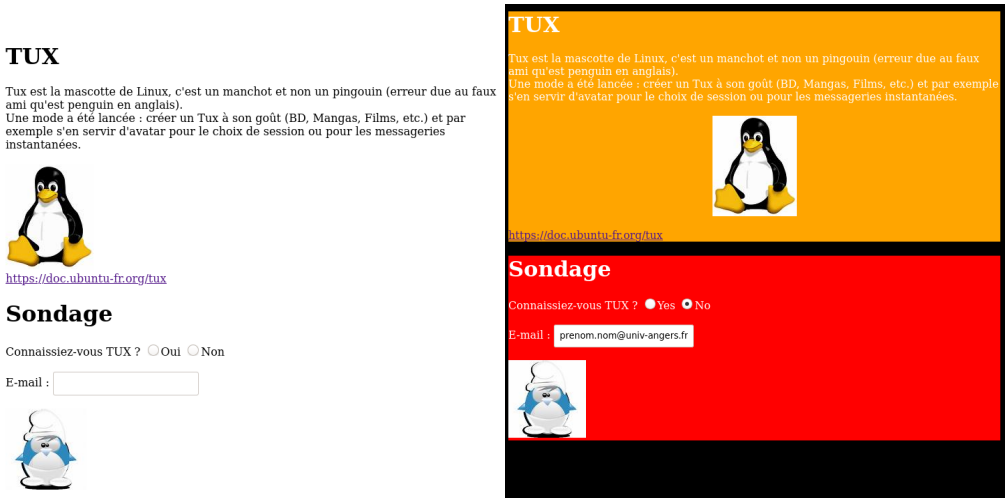

FIGURE 1 – Modification statique d'un document HTML. Vue de gauche : document original (**tux.html** seul). Vue de droite : document avec les modifications (**tux.html + tux.js**).

1. Modifiez la couleur d'arrière plan du corps du document (`<body>`) en noir. Modifiez les couleurs d'arrière plan des divisions (`<div>`) d'identifiant `intro` et `sondage`, respectivement, en orange et en rouge. Modifiez la couleur des éléments de chaque division du document en blanc.
2. Centrez **toutes** les images de la division d'identifiant `intro`. Pour ce faire, vous pouvez utiliser la classe CSS `center` définie dans **tux.html**.
3. Dans la division `sondage` :

- **Tous** les champs de saisie d'un email doivent prendre comme valeur *nom.prenom@univ-angers.fr*.
- **Toutes** les cases de valeur `non` doivent être cochées.
- Les labels de **toutes** les cases de valeur `oui` et `non` doivent être, respectivement, *Yes* et *No*.

4. Ajoutez pour chaque image une description textuelle (`alt`) indiquant où se situe l'image (arborescence DOM jusqu'à l'élément `body`). Par exemple pour la première image : `alt="Image - P (parIntro) - DIV (intro)"` indique que l'image est dans le paragraphe (nom de l'étiquette : P) d'identifiant `parIntro` qui est dans la division (nom de l'étiquette : DIV) d'identifiant `intro`.

Exercice 4. (**Évènements et Formulaires**)
Dans cet exercice, on vous demande de réaliser un système de vérification de formulaire. La page permet de réaliser une enquête de satisfaction sur le cours de Développement Web, avec la partie PHP et la partie JS. La page web est constituée de trois fichiers **satisfaction.html** qui contient la structure de la page et que vous ne devez pas modifier, **satisfaction.css** pour gérer l'affichage des éléments du formulaire et que vous ne devez pas modifier non plus, **satisfaction.js** qui devra contenir votre code pour vérifier le formulaire.
1. Pour être valide, toutes les informations sur le participant doivent être renseignés. Modifier la fonction `participantValid` qui doit retourner si tous les `input` du "Participant" sont remplis.
2. Pour être valide, au moins une valeur des boutons radios doit être sélectionnée. Modifier la fonction `radioValid` qui doit retourner s'il existe un bouton radio identifié par le nom passé en paramètre coché.
3. Pour être valide, l'ordre de préférence donné par l'utilisateur doit être renseigné, et bien construit. On n'autorise pas les "égalités" et donc chaque partie de cours doit avoir une valeur différente des autres parties. Modifier la fonction `sortValid` qui doit retourner si les `select` identifiés par la classe passée en paramètre sont bien renseignés.
4. Pour être valide, tous les champs du formulaire devront être bien remplis. Si une partie du formulaire n'est pas bien rempli, un message d'erreurs doit être affiché à l'utilisateur via une fenêtre de dialogue, sinon le formulaire est transmis. Modifier la fonction `submitForm` pour gérer la soumission du formulaire. Sans modifier le fichier HTML, vous ferez en sorte que la soumission du formulaire soit gérée par la fonction `submitForm`.

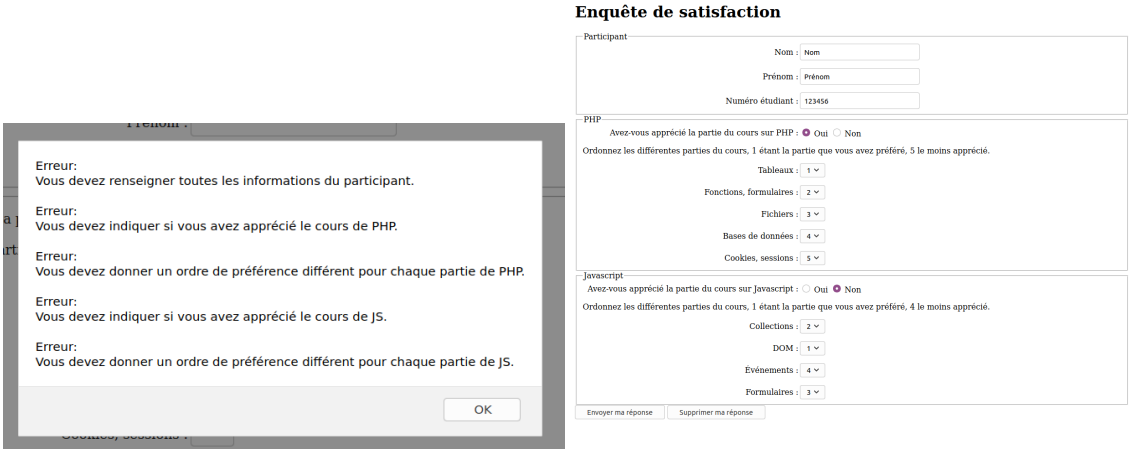

FIGURE 2 – Captures de la page de satisfaction. Capture de gauche : Affichage du message d'erreur où aucun champ n'est correctement renseigné. Capture de droite : Page contenant des données bien remplies.